Ask the Airport

Build a generative AI application in one afternoon. Bring an idea, pick your stack, ship a public repo. The more of the platform you use, the more points you earn — but nothing stops you competing.

DATE	SPRINT	TEAMS	DEMO	POINTS
Wed 02 Sep 2026	**2h30**	**10 · 4-5 people**	**2 min max**	**100**

READ THIS FIRST

The event runs on **Wednesday, 2 September 2026**, in São Paulo. Everything issued on the day — AWS account, Bedrock key, EC2 instance and S3 prefix — is **per team**, shared by everyone on it.

Mind the region split: the **AWS console and your EC2 are in São Paulo** (`sa-east-1`), and **Bedrock is in Singapore** (`ap-southeast-1`). That is deliberate, not a bug.

The challenge

An AI application. That's the brief — the rest is yours.

We're not prescribing the problem. Bring a perspective: an assistant, an analytics tool, a copilot, an agent, something we haven't thought of. The judging rewards clever ideas and genuine business value far more than it rewards checking boxes.

Directions worth stealing

- **Natural-language analytics** — a question in, SQL generated, answer out.
- **Disruption copilot** — cross-reference flights and weather to predict which connections break, and explain why in plain language.
- **Travel agent with memory** — learn a passenger's preferences in one conversation, apply them in the next. Rebook them when their flight slips three hours.
- **Semantic concierge** — vector search over routes and notes for “find me something like this”.
- **Something else entirely.** Genuinely — the creativity points are real.

The airportdb dataset

The suggested playing field. You don't have to use it.

We've prepared a realistic airline database — airports, flights, bookings, passengers and weather, filtered to flights touching Brazil across one week in June 2015. It's small enough to import in minutes and rich in interesting problems: delays cascade, connections break, weather disrupts, passengers have preferences and histories. Teams that find a real problem in this data and solve it well will have an easier time scoring on creativity and business value than teams starting from a blank sheet.

ITEM	DETAIL
Window	2015-06-02 to 2015-06-09 (one week)
Scope	All flights touching a Brazilian airport, plus a global sample for variety
Size	5,191 flights · 617,062 bookings · 36,095 passengers · weather included
Tables	12 in total: `airline` `airplane` `airplane_type` `airport` `airport_geo` `booking` `employee` `flight` `flightschedule` `passenger` `passengerdetails` `weatherdata`
Download	`hackaton-tidb.s3.sa-east-1.amazonaws.com/dumps/hackathon_airportdb.sql.gz` — 10 MB gzipped

How points work

100 points, split between what you built and how you built it. Part A is judged live from your demo. Part B is verified by AI reading your public repo.

TO BE JUDGED, YOU NEED

A **public GitHub repo**, a **SUBMISSION.md** at its root, **TiDB used somewhere** in your application, and a **working demo of 2 minutes or less**. That's the whole floor. Everything else is points, not gates.

Part A — what you built

JUDGED LIVE FROM YOUR DEMO

60 pts

20 **It works**
The thing runs and does what you say it does. A rough app that works beats a beautiful one that doesn't.

15 **Innovation and creativity**
An angle we haven't seen four times already today. Surprise us.

15 **Business value**
Who has this problem, and is this a real answer to it? Name the user.

10 **Demo and pitch**
Two minutes, one clear story. Show the thing working rather than describing it.

Part B — how you built it

VERIFIED BY AI FROM YOUR PUBLIC REPO

40 pts

10 **TiDB Cloud Starter on AWS**
Your data lives in a TiDB Cloud cluster in AWS São Paulo (`sa-east-1`). Running TiDB locally instead still keeps you eligible — it just doesn't score here.

8 **TiDB vector search**
A `VECTOR` column queried with `VEC_COSINE_DISTANCE`, or the `EMBED_TEXT` auto-embedding function. Both count.

Part B — how you built it

VERIFIED BY AI FROM YOUR PUBLIC REPO

40 pts

8 **Amazon Bedrock**

Used for text generation, embeddings, or both. Any other LLM API keeps you fully eligible — you just don't score these 8.

8 **Deployed on AWS**

The deploy target is your team's EC2 instance in `sa-east-1`. Any other AWS service reachable at a URL counts too. Running on your laptop is fine; it just scores zero here.

6 **Built with Kiro**

Commit your `.kiro/` specs and steering files so we can see the spec-driven work, not just the output.

THE HONEST MATH

A team that runs everything locally with no AWS account at all can still score **60 points plus vector search**. A team that uses the whole platform tops out at 100. The gap is real but it is not a wall — a brilliant local build will beat a dull deployed one.

What you get on the day

Everything below is per team, not per person. One account, one key, one machine, one prefix — shared by the 4 or 5 of you.

RESOURCE	WHAT YOU GET
AWS account	An IAM user in the form `latam-hackathon-0XX`, signing in at `tidb-latam-hackathon.signin.aws.amazon.com/console`. **Enroll MFA on first sign-in** — the Bedrock API key is only released after that.
Bedrock key	One key per team, issued for `ap-southeast-1` (Singapore). Never share credentials between teams.
EC2 instance	One per team in `sa-east-1`, tagged with the team name. Access via Session Manager, no SSH key needed. Ports `8000-8999` and `3000` are open inbound — bind your app to `0.0.0.0`, never `127.0.0.1`.
S3 prefix	One shared bucket, one prefix per team: `s3://tidb-latam-hackathon-2026-048364544505/latam-hackathon-0XX/`
TiDB Cloud	Not issued: each team registers its own free Starter cluster at `tidbcloud.com`. The organisers never hold those credentials.
Kiro	Not issued. **Install it before you arrive**, from `https://kiro.dev`, and sign in. Credits are handed out on the day; the install shouldn't eat your sprint.

BEFORE YOU ARRIVE (TWENTY MINUTES AT HOME)

GitHub account · TiDB Cloud account (free, no card) · Python 3.9+ and pip working, or the language of your choice · Kiro installed and signed in.

Your team's EC2 public IP changes on every stop/start — there is no Elastic IP. Don't hard-code it anywhere and, above all, **don't restrict TiDB Cloud access to it**: you'd lose your database mid-sprint. The Starter's default access setting is fine for a throwaway cluster over 2h30.

SHELF LIFE

After **2026-09-05**, all AWS accounts, EC2 instances, S3 data, TiDB clusters and API keys are deleted. Take what matters home before then — your public repo is what survives.

Bedrock in Singapore

The console and your EC2 are in São Paulo. Bedrock is in Singapore. This is expected.

Your team's Bedrock key was issued for `ap-southeast-1`. Point the client at `sa-east-1` — or any other region — and you'll get an authentication or access-denied error, not an error that says “wrong region”. Remember that symptom: **a Bedrock credential error is almost always the wrong region**. Start by copying the `.env` below, which already has the region right.

Copy-paste .env

```
# Bedrock — Singapore
AWS_BEARER_TOKEN_BEDROCK=<your-team-key>
AWS_REGION=ap-southeast-1

# TiDB Cloud Starter — São Paulo
TIDB_HOST=<host>.clusters.tidb-cloud.com
TIDB_PORT=4000
TIDB_USER=<user>
TIDB_PASSWORD=<password>
TIDB_DATABASE=airportdb
```

PURPOSE	MODEL ID AVAILABLE IN ap-southeast-1
Text	`anthropic.claude-3-5-sonnet-20240620-v1:0`
Text (faster)	`anthropic.claude-3-haiku-20240307-v1:0`
Embeddings	`cohere.embed-english-v3` — 1024 dimensions
Multilingual embeddings	`cohere.embed-multilingual-v3` — 1024 dimensions

Python client, region already correct

```
import json, os, boto3
from dotenv import load_dotenv

load_dotenv()

bedrock = boto3.client("bedrock-runtime", region_name="ap-southeast-1")

def ask(prompt: str) -> str:
    resp = bedrock.invoke_model(
        modelId="anthropic.claude-3-haiku-20240307-v1:0",
        body=json.dumps({
            "anthropic_version": "bedrock-2023-05-31",
            "max_tokens": 500,
            "messages": [{"role": "user", "content": prompt}],
        }),
    )
    return json.loads(resp["body"].read())["content"][0]["text"]
```

TWO DETAILS THAT SAVE YOU TIME

Titan and Mistral do not exist in `ap-southeast-1`. Copy a `modelId` from a tutorial and, if it isn't in the table above, you'll get `ValidationException`.

The latency to Singapore is real: a few hundred milliseconds per call, round trip. Batch the work into a few large calls; don't chain Bedrock inside a row-by-row loop.

TiDB Cloud and vector search

Cluster to first query in about ten minutes.

1. Create the cluster

At `tidbcloud.com`, create a new resource. Choose the **Starter** plan (free), Cloud Provider **AWS** and Region **São Paulo (sa-east-1)**. It's active in one to two minutes. The free tier gives you 5 GiB and 50M request units per month — far more than this dataset needs.

Open the cluster, click **Connect**, keep **Public Endpoint** and copy the host, port (4000), username and database name. The password shows once — save it now or reset it later. Never commit those credentials: the repo is public.

2. Load the dataset

```
curl -O https://hackaton-tidb.s3.sa-east-1.amazonaws.com/dumps/hackathon_airportdb.sql.gz
gunzip hackathon_airportdb.sql.gz

mysql -h <HOST> -P 4000 -u <USERNAME> -p \
  --ssl-mode=VERIFY_IDENTITY --ssl-ca=/etc/ssl/cert.pem \
  airportdb < hackathon_airportdb.sql
```

The public endpoint requires TLS — that's what `--ssl-ca` is for. On the venue wifi, prefer TiDB Cloud's **import from S3** in the console: the cluster pulls the file server-side and never touches your connection.

3. Connect from Python

```
pip install pymysql boto3 python-dotenv
```

```
import os, pymysql
from dotenv import load_dotenv

load_dotenv()

conn = pymysql.connect(
    host=os.environ["TIDB_HOST"],
    port=4000,
    user=os.environ["TIDB_USER"],
    password=os.environ["TIDB_PASSWORD"],
    database="airportdb",
    ssl={"ca": "/etc/ssl/cert.pem"},
)

with conn.cursor() as cur:
    cur.execute("SELECT COUNT(*) FROM flight")
    print(cur.fetchone()[0], "flights loaded")
```

4. Vector search the short way: EMBED_TEXT

`EMBED_TEXT()` is TiDB's own and **has nothing to do with Bedrock**: no key, no region, no SDK. You insert text and TiDB generates the vectors. It's the cheapest route to the 8 vector-search points.

```
CREATE TABLE flight_notes (
  id BIGINT AUTO_RANDOM PRIMARY KEY,
  flight_id INT,
  note TEXT,
  embedding VECTOR(1024) GENERATED ALWAYS AS (
    EMBED_TEXT("tidbcloud_free/amazon/titan-embed-text-v2", note)
  ) STORED
);

INSERT INTO flight_notes (flight_id, note)
VALUES (1, 'direct flight from Sao Paulo to Frankfurt');

SELECT flight_id, note
FROM flight_notes
ORDER BY VEC_COSINE_DISTANCE(embedding, EMBED_TEXT(
  "tidbcloud_free/amazon/titan-embed-text-v2", 'nonstop to Germany'))
LIMIT 5;
```

If you'd rather generate the embeddings yourself, use `cohere.embed-multilingual-v3` on Bedrock — also 1024 dimensions, so the `VECTOR(1024)` column definition stays the same.

Spending 2h30

A shape that works. Adjust it, but protect the last fifteen minutes — teams that skip them don't submit.

`0:00 – 0:15`	**Pick the idea and split up**	Decide the one thing your demo will show. Assign: data, AI, interface, pitch.
`0:15 – 0:30`	**Cluster up, data in**	One person only. Everyone else starts building against fake data.
`0:30 – 1:45`	**Build the core loop**	Input to answer, end to end, ugly. Make it work before making it more.

1:45 – 2:05	**Grab the stack points**	Vector column, deploy to EC2, commit your .kiro/ specs. Cheap points, quick wins.
2:05 – 2:20	**Write SUBMISSION.md, push public**	Non-negotiable. Check the repo is actually public.
2:20 – 2:30	**Record the demo, rehearse the line**	Two minutes. Show, don't narrate.

HOW A TEAM OF FIVE AVOIDS GRIDLOCK

One laptop drives, the team steers. One person owns the cluster and the data load while everyone else works against fake data — five people watching the same `mysql <` finish is half an hour thrown away.

Submitting

A public GitHub repo, plus a comment on the pinned “Entregas” issue. That's it.

The submission, in four steps

1. Build in **your own team's public repo** on GitHub. Don't fork the main repository and don't open a Pull Request there.
2. The repo must contain the code, your `.kiro/` specs if you used Kiro, and a `SUBMISSION.md` at the root: what you built, how to run it, what you'd do next.
3. When you're done, **comment on the pinned “Entregas” issue** in the main repository with three things: team name, repo URL and demo link — 2-minute video or live URL.
4. That's it. That's the submission.

The freeze

At the deadline, the organisers **fork every submitted repository**. That fork is what goes to judging — and a fork does not sync. Anything you push after the deadline simply isn't evaluated.

TWO MINUTES BEFORE YOU COMMENT

Push everything first, comment second. Whatever isn't on `main` at the deadline doesn't exist as far as the judges are concerned.

Confirm the repo is genuinely public. A private repo can't be forked and therefore can't be scored — the silliest way there is to lose 100 points.

By submitting, your team agrees that the organisers keep an archived copy of the repository — the fork.

The SUBMISSION.md file

Create `SUBMISSION.md` in your repository root and fill it in. Copy this template exactly — the AI evaluator reads it first, then checks your claims against your actual code.

```
# SUBMISSION.md

## Team
Team name:
Members:

## Pitch
One line on what this does and who it's for.

## What it does
Two or three sentences. What problem, what approach, what the user sees.

## Stack — check what you actually used
- [ ] TiDB Cloud Starter on AWS sa-east-1
- [ ] TiDB vector search (VECTOR column or EMBED_TEXT)
- [ ] Amazon Bedrock (ap-southeast-1)
- [ ] Deployed on AWS  -> live URL:
- [ ] Built with Kiro (.kiro/ committed)

## Where to look
TiDB connection/queries:  path/to/file.py
Vector search:            path/to/file.py
Bedrock calls:            path/to/file.py

## Demo
Link to your 2-minute video or live app:
```

ONE RULE ABOUT THE CHECKBOXES

Claims are **verified against your code**, not taken on trust. A ticked box our evaluator can't find in the repository scores zero for that line. Pointing us at the right file in “Where to look” is the fastest way to get credit for what you actually did.

Put your secrets in `.env` and `.env` in `.gitignore` — the repo is public. Commit a `.env.example` instead.

When something breaks

Every failure below has a route around it that keeps you in the competition.

SYMPTOM	FIX, OR ROUTE AROUND IT
`AccessDeniedException` on Bedrock	Check the region before anything else: the key is for `ap-southeast-1`. If the region is right, the Anthropic first-use form hasn't been submitted for the account — open any Anthropic model in the Bedrock console and complete it. Still blocked? Swap to another LLM API and carry on: it costs 8 points, nothing else.
`ValidationException`: model not supported	That model doesn't exist in `ap-southeast-1`. Use Claude 3.5 Sonnet, Claude 3 Haiku or the Cohere embeddings. Titan and Mistral aren't there.
Bedrock call is too slow	That's the distance to Singapore. Pack several items into one prompt instead of calling the model inside a loop.
SSL error connecting to TiDB	The public endpoint requires TLS. In Python, `ssl={"ca": "/etc/ssl/cert.pem"}`; on the CLI, `--ssl-ca`.
TiDB connection times out	Check Network Access on the cluster. The Starter default is fine — if someone restricted it by IP, undo that: the EC2 public IP changes on every stop/start and will drop your access mid-sprint.

SYMPTOM	FIX, OR ROUTE AROUND IT
App on EC2 won't open in the browser	You probably bound to `127.0.0.1`. Bind to `0.0.0.0` and use a port in `8000-8999` or `3000` — those are the only ones open. Check the current public IP too; it changes on every restart.
Dataset import crawling	Venue wifi. Use import-from-S3 in the TiDB Cloud console instead: the cluster fetches the file directly.
No access to the AWS account	Build locally with any LLM API. You keep all 60 Part A points and can still score vector search via `EMBED_TEXT`.
Out of time to deploy	Then don't. Run locally, record the demo, submit. Eight points is not worth a missed submission.

> IF YOU'RE PROPERLY STUCK
>
> Ask a mentor before you burn fifteen minutes on it. Nobody wins points for debugging alone, and there is always a route that keeps you scoring.